Homework 3 — Indicizzazione e ricerca di file .txt con Lucene

Relazione: analyzer, costruzione del dataset, tempi di indicizzazione e valutazione con metriche

STUDENTE	MATRICOLA	EMAIL	TECNOLOGIE
Alessio Pecilli	589232	ale.pecilli@stud.uniroma3.it	Java 21+ · Maven · Lucene 10.1.0

Repository: https://github.com/Alessio-Pecilli/IDD-HW3

1 Struttura del progetto

Indexer.java
Crea un `Document` per ogni file `.txt` (campi `nome`, `contenuto`, `path`) e lo aggiunge con un `IndexWriter`. Cartella dei documenti e dell'indice da riga di comando.

Searcher.java
Legge le query da console (`nome …` / `contenuto …`), le passa al `QueryParser` e stampa score, path e tempo di risposta dei primi 10 risultati.

Valutazione.java · Metriche.java
Eseguono le query giudicate su tutta la collezione e calcolano le metriche; confrontano tre analyzer per il campo `contenuto`.

Benchmark.java
Misura il tempo di indicizzazione su sottoinsiemi crescenti della collezione e il tempo medio di risposta delle query.

scripts/build_dataset.py
Scarica le voci di Wikipedia e scrive i file `.txt` e il manifest con la ground truth (§2.2).

src/test, scripts/test_*.py
52 test automatici (JUnit + unittest), descritti in §7.

```
python3 scripts/build_dataset.py --max-voci 2300   # dataset-wiki/ + eval/manifest.tsv
mvn compile exec:java -Dexec.mainClass=it.uniroma3.idd.Indexer -Dexec.args="dataset-wiki index"
mvn exec:java -Dexec.mainClass=it.uniroma3.idd.Searcher -Dexec.args="index"
mvn exec:java -Dexec.mainClass=it.uniroma3.idd.Benchmark     # -> eval/benchmark.md
mvn exec:java -Dexec.mainClass=it.uniroma3.idd.Valutazione   # -> eval/risultati.md, eval/
top10.tsv
mvn test && python3 -m unittest discover -s scripts       # test automatici
```

2 Costruzione dei dataset

Si usano due collezioni con scopi diversi: una piccola, scritta a mano, per verificare il comportamento di ogni funzionalità con risultati prevedibili, e una grande, presa da Wikipedia, per misurare i tempi e valutare l'efficacia con le metriche.

2.1 Dataset piccolo (`dataset/`, 12 file)

I 12 file sono stati scritti a mano: 2–3 frasi ciascuno (circa 190 caratteri) su argomenti diversi (Roma antica, cucina, astronomia, motori di ricerca, Lucene…). Ogni file è stato costruito per mettere alla prova un aspetto preciso:

- **nomi con separatori diversi**: `storia_di_roma` (underscore), `motori-di-ricerca` (trattino), `Ricetta Carbonara` (spazio e maiuscola), per l'analyzer del campo `nome`;

- **forme flesse** (*pianeta*/*pianeti*, *romano*/*romana*/*romane*) ed **elisioni** (*l'impero*, *dell'informazione*), per lo stemming e l'`ElisionFilter`;
- **frasi note** (*selva oscura*, *indice invertito*), per le phrase query;
- **ambiguità volute**: la carbonara contiene *pecorino romano* e il calcio contiene *Roma*, così una query su *impero romano* restituisce anche documenti non pertinenti, utili per controllare il ranking.

Con 12 documenti i risultati di ogni query si conoscono in anticipo, quindi questa collezione è usata dai test JUnit (§7). Per la stessa ragione però non basta a valutare tempi e recall.

2.2 Dataset grande (`dataset-wiki/`, 1980 voci di Wikipedia)

Per avere una collezione realistica e, allo stesso tempo, sapere quali documenti sono rilevanti per una query, il dataset è stato costruito dall'API di *it.wikipedia.org* con lo script `scripts/build_dataset.py` in tre passi:

1. **Voci tematiche.** Per 16 categorie di Wikipedia scelte su argomenti diversi tra loro (tabella sotto) si scaricano tutte le voci che appartengono *direttamente* alla categoria (`list=categorymembers`, namespace 0, senza sottocategorie). L'appartenenza alla categoria è decisa dagli autori di Wikipedia, non da me: è quindi una **ground truth indipendente** dal sistema da valutare.
2. **Voci casuali.** Si aggiungono 3000 voci estratte a caso (`list=random`), che fanno da "rumore": allargano la collezione e contengono documenti che condividono termini con le query senza essere pertinenti (es. un calciatore di *Roma*, un film intitolato *Inferno*).
3. **Testo.** Il testo di ogni voce è scaricato come wikitext, a blocchi di 50 voci per richiesta per rispettare i limiti dell'API, e convertito in testo in chiaro: vengono tolti template, tabelle, note, immagini e markup, e la voce è troncata alle sezioni finali (*Note*, *Bibliografia*, *Collegamenti esterni*…), che contengono quasi solo link. Le voci con meno di 300 caratteri (abbozzi, redirect) sono scartate. Ogni voce diventa un file `<titolo>.txt`: il campo `nome` contiene quindi il titolo della voce, spesso con un disambiguatore tra parentesi, ad esempio `Io (astronomia)`.

Lo script scrive anche `eval/manifest.tsv`, che per ogni file riporta `pageid`, `revid` (la revisione esatta scaricata), lunghezza e codici delle categorie di appartenenza. Il manifest è nel repository, i 12 MB di testo no (`.gitignore`): si rigenerano con lo script.

Dimensione finale. Le liste delle categorie e delle voci casuali contengono 4229 voci. Per limitare il numero di richieste all'API (che rispondeva spesso *HTTP 429 Too Many Requests*) sono state scaricate le prime 2300 in ordine di `pageid`. Tolte quelle sotto i 300 caratteri restano **1980 documenti**: 923 tematici e 1057 casuali, per 12,1 milioni di caratteri di testo (12,2 MB in UTF-8). Lunghezza: minimo 300, mediana 2429, media 6126, massimo 160 481 caratteri. Per questo di alcune categorie è presente solo una parte delle voci (colonna "Nel dataset"). La ground truth è calcolata solo sui documenti presenti, quindi resta coerente.

Codice	Categoria Wikipedia	Voci nella categoria	Nel dataset
`IMP`	Imperatori romani	103	102
`GIO`	Satelliti di Giove	92	72
`SAT`	Satelliti di Saturno	77	61
`INF`	Canti dell'Inferno	34	34
`PUR`	Canti del Purgatorio	33	33
`PAR`	Canti del Paradiso	33	33
`MOT`	Motori di ricerca	111	73
`LIN`	Linguaggi di programmazione	144	105
`ALG`	Algoritmi di ordinamento	30	27
`DB`	Basi di dati	59	28
`DOG`	Dogi della Repubblica di Venezia	129	127
`PIT`	Pittori italiani del Rinascimento	239	133
`PUN`	Battaglie della seconda guerra punica	55	27
`ELE`	Elementi chimici	57	53
`FOR`	Formaggi italiani	22	6
`PAS`	Tipi di pasta	16	9
	Totale tematiche + 1057 casuali	**1234**	**923 + 1057 = 1980**

La colonna "Voci nella categoria" somma 1234 perché due voci appartengono a due categorie (CiteSeer: DB e MOT; XBase: DB e LIN), quindi le voci tematiche distinte sono 1232. Il manifest ammette più codici per documento, ma nessuna delle due è tra le voci scaricate.

3 Analyzer

Gli analyzer sono definiti per campo con un `PerFieldAnalyzerWrapper`, restituito da `Indexer.analyzer()` e usato sia in indicizzazione sia dal `QueryParser`. In questo modo query e documenti producono termini compatibili.

Campo nome — `CustomAnalyzer`
`WhitespaceTokenizer` → `LowerCaseFilter` → `WordDelimiterGraphFilter`

- `storia_di_roma` → *storia, di, roma*; `Io (astronomia)` → *io, astronomia*: il filtro spezza su `_ - . ( )` (i cambi maiuscolo/minuscolo non contano, perché il testo è già in minuscolo);
- niente stopword né stemming: il nome è breve, ogni parola conta e deve funzionare la frase `"motori di ricerca"`;
- l'estensione `.txt` finale viene tolta prima dell'indicizzazione.

Campo contenuto — `ItalianAnalyzer`
`StandardTokenizer` → elisione → minuscolo → stopword → stemming leggero

- *L'Impero Romano* → *imper, roman*; *il pianeta e i pianeti* → *pianet, pianet*;
- la scelta è verificata sperimentalmente in §6.3, confrontandola con `StandardAnalyzer` e con lo stesso analyzer italiano senza stemming.

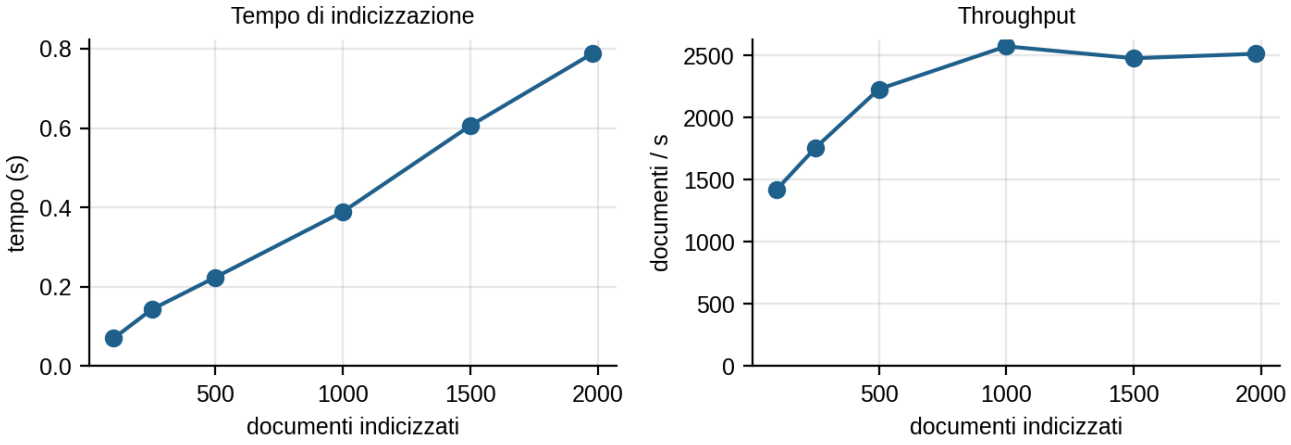
4 Indicizzazione e tempi

Il tempo è misurato da `Benchmark.java` su sottoinsiemi casuali (seed fisso) della collezione Wikipedia. Comprende apertura della `Directory`, creazione dell'`IndexWriter`, lettura dei file, analisi, `addDocument` e `commit()`. Prima delle misure si fa un'esecuzione di riscaldamento della JVM su 500 documenti, che viene

scartata; per ogni dimensione si riporta poi la mediana di 5 esecuzioni (codice compilato per Java 21, eseguito su Java 25, 8 core).

Documenti	Testo (MB)	Tempo (ms)	Documenti/s	MB/s	Indice su disco (MB)
12 (dataset piccolo)	0,002	~450*	–	–	< 0,01
100	0,5	70	1423	7,6	0,2
250	1,3	143	1754	9,2	0,5
500	2,9	224	2228	12,9	1,1
1000	5,9	389	2573	15,2	2,0
1500	9,4	606	2477	15,6	3,0
1980 (tutto)	**12,2**	**788**	**2513**	**15,5**	**3,8**

* Misura a parte, non prodotta da `Benchmark.java`: singola esecuzione di `Indexer` a JVM "fredda", dominata dall'avvio di Lucene e non dai documenti.



Tempo di indicizzazione e throughput al crescere della collezione.

- **Andamento lineare.** Da 1000 documenti in su il tempo cresce in modo proporzionale ai documenti e il throughput si stabilizza a circa 2500 documenti/s (15 MB/s di testo). Sotto i 500 documenti pesano i costi fissi (apertura dell'indice, `commit()`, creazione dei segmenti), per questo il throughput è più basso. Con i soli 12 file del dataset piccolo si misurano solo questi costi fissi.
- **Dimensione dell'indice.** L'indice occupa circa il 31% del testo (3,8 MB su 12,2 MB). Il contenuto infatti non è memorizzato (`Field.Store.NO`): l'indice contiene solo dizionario e posting list, oltre a nome e path memorizzati.
- **Tempo di risposta.** Sull'indice completo le 50 query di valutazione (primi 10 risultati, 50 ripetizioni ciascuna) richiedono in media **0,18 ms** (mediana 0,15 ms).

5 Sintassi delle query e query di test funzionali

La prima parola indica il campo (`nome` o `contenuto`), il resto è passato al `QueryParser`: termini in OR, `+termine` obbligatorio, `-termine` escluso, `AND`/`OR`/`NOT`, `"…"` per le phrase query. Il ranking è BM25 (default di Lucene). Sul dataset piccolo i risultati sono noti in anticipo, quindi le query di prova sono verificate automaticamente da `SearcherTest`: **20 test con 25 query** in tutto. La tabella ne riporta una **selezione di 12**, scelta per coprire tutte le funzionalità; le altre sono elencate sotto la tabella. Le 50 query usate per la valutazione con le metriche sono invece in §6 e in Appendice A.

#	Query	Query Lucene	Risultati	Cosa verifica
1	`nome roma`	`nome:roma`	storia_di_roma	split su `_`

2	`nome carbonara`	`nome:carbonara`	Ricetta Carbonara	maiuscole e spazio
3	`nome "motori di ricerca"`	`nome:"motori di ricerca"`	motori-di-ricerca	phrase query sul nome
4	`nome "ricerca motori"`	`nome:"ricerca motori"`	nessuno	nella frase l'ordine conta
5	`nome pizze`	`nome:pizze`	nessuno	sul nome niente stemming
6	`contenuto pianeti`	`contenuto:pianet`	sistema_solare	stemming (= `pianeta`)
7	`contenuto "selva oscura"`	`contenuto:"selva oscur"`	divina_commedia	phrase query
8	`contenuto +guanciale +pecorino`	`+contenuto:guancial +contenuto:pecorin`	Ricetta Carbonara	termini obbligatori
9	`contenuto romano -pecorino`	`contenuto:roman -contenuto:pecorin`	impero-romano, storia_di_roma	esclusione
10	`contenuto impero romano`	`contenuto:imper contenuto:roman`	impero-romano > storia_di_roma > Ricetta Carbonara	ranking BM25
11	`contenuto il di la`	(vuota)	nessuno	solo stopword
12	`titolo roma` · `contenuto "frase`	–	errore	campo non valido · sintassi errata

Altre query verificate da `SearcherTest` (non in tabella): `nome romano` (split su `-` → impero-romano), `nome CARBONARA` (query in maiuscolo), `nome roma solare` (OR sul nome → 2 risultati), `NOME roma` (nome del campo case-insensitive), `nome  roma` (spazi multipli), `contenuto pianeta` (stessi risultati di *pianeti*), `contenuto impero` (elisione: trova *l'impero*), `contenuto lucene AND java` (AND esplicito), `contenuto romano` (senza esclusione include la carbonara), `contenuto xyzzy` (termine assente → nessun risultato), `contenuto` senza termini (errore).

6 Valutazione dell'efficacia

6.1 Metodologia

Query e giudizi di rilevanza. In `eval/queries.tsv` ci sono 50 query (elencate tutte in Appendice A), ciascuna associata a uno o più argomenti (codici della tabella §2.2). Un documento è **rilevante** per la query se nel manifest appartiene a uno di quegli argomenti. Per ogni argomento ci sono più formulazioni: parole chiave sciolte, frase esatta, termini obbligatori/esclusi e query sul campo `nome`. In questo modo si vede anche l'effetto della formulazione. Le query sono state scritte prima di guardare i risultati e non sono state modificate per migliorarli.

Esecuzione. Ogni query è eseguita sull'intera collezione e si considera *tutto* il ranking, non solo i primi 10 risultati: altrimenti la recall non si potrebbe calcolare. La rilevanza è binaria.

Metriche (implementate in `Metriche.java`, verificate con esempi calcolati a mano in `MetricheTest`). R è l'insieme dei rilevanti, *rel(i)* vale 1 se il documento in posizione i è rilevante:

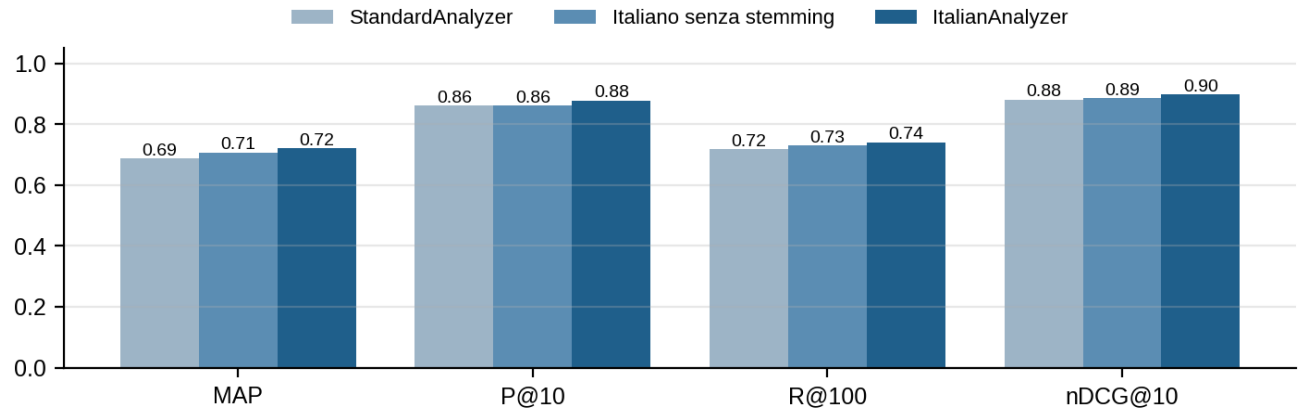
Metrica	Definizione	Cosa misura
P@10	$\lvert rilevanti\ tra\ i\ primi\ 10 \rvert\ /\ 10$	qualità della prima pagina, quella che l'utente guarda
R@100	$\lvert rilevanti\ tra\ i\ primi\ 100 \rvert\ /\ \lvert R \rvert$	quanti rilevanti si trovano scorrendo 100 risultati
Recall	$\lvert rilevanti\ restituiti \rvert\ /\ \lvert R \rvert$	copertura dell'intero insieme restituito (tutti i documenti con almeno un termine)
R-Precision	$P@\lvert R \rvert$	precision nel punto in cui precision = recall

AP / MAP	$(1/\|R\|)\,\Sigma_{i:\,rel(i)=1}\,P@i$	qualità di tutto il ranking; i rilevanti non trovati contano 0. MAP = media su tutte le query
RR / MRR	$1 / posizione\ del\ primo\ rilevante$	quanto presto compare il primo risultato utile
nDCG@10	$\Sigma_{i \le 10}\, rel(i)/log_2(i+1) \,/\, IDCG@10$	come P@10, ma pesa di più le prime posizioni

6.2 Risultati

Medie sulle 50 query (dettaglio completo delle 50 query in Appendice A e in `eval/risultati.md`). La riga evidenziata è la configurazione scelta. Le query sul campo `nome` sono identiche nelle tre configurazioni, perché cambia solo l'analyzer del contenuto.

Analyzer contenuto	MAP	P@10	R@100	Recall	R-Prec	MRR	nDCG@10
StandardAnalyzer	0,687	0,862	0,717	0,762	0,667	0,940	0,881
Italiano senza stemming	0,707	0,862	0,729	0,780	0,689	0,953	0,886
ItalianAnalyzer (scelto)	**0,720**	**0,878**	**0,741**	**0,792**	**0,699**	**0,943**	**0,897**



Confronto delle tre configurazioni per il campo contenuto.

Una selezione di query (`ItalianAnalyzer`) mostra come la formulazione cambia il rapporto tra precision e recall:

Id	Query	|R|	Restituiti	P@10	Recall	AP	nDCG@10
q01	`contenuto imperatore romano`	102	390	1,000	0,990	0,923	1,000
q18	`contenuto "imperatore romano"`	102	37	0,800	0,324	0,276	0,842
q22	`contenuto +imperatore +impero +augusto`	102	71	1,000	0,627	0,621	1,000
q12	`contenuto doge Venezia`	127	235	1,000	0,976	0,960	1,000
q41	`contenuto "doge di Venezia"`	127	8	0,700	0,055	0,055	0,801
q21	`nome doge`	127	11	0,900	0,079	0,078	0,936
q19	`nome inferno`	34	34	1,000	1,000	1,000	1,000
q05	`contenuto canto Inferno Dante`	34	178	0,600	1,000	0,647	0,691
q11	`contenuto database`	28	62	0,700	0,750	0,395	0,556
q17	`contenuto pasta`	9	15	0,600	0,889	0,811	0,777
q48	`contenuto "formaggio a pasta"`	6	0	0,000	0,000	0,000	0,000

6.3 Discussione

- **Effetto dello stemming.** L'`ItalianAnalyzer` migliora tutte le metriche medie rispetto a `StandardAnalyzer` (MAP +0,033, Recall +0,030). Il guadagno è grande sulle query restrittive, dove una forma flessa diversa basta a escludere un documento: `+imperatore +impero +augusto` passa da AP 0,235 a 0,621 (*imperatore*/*imperatori* → *imperator*), `canto Inferno Dante` da 0,400 a 0,647 (*canto*/*canti*), la frase `"imperatore romano"` da 0,134 a 0,276. Sulle query OR molto ampie (`indice ricerca informazioni documenti`) lo stemming restituisce più documenti e perde invece qualche centesimo di AP. Le stopword e l'elisione da sole (riga intermedia) danno già circa il 60% del miglioramento della MAP.
- **Precision contro recall.** Le query OR di 2–4 parole hanno recall vicina a 1 e P@10 alta grazie al ranking BM25. Le phrase query hanno precision alta ma recall molto bassa: `"doge di Venezia"` restituisce 8 voci, di cui solo 7 dogi su 127, perché quasi tutte le voci scrivono *doge della Repubblica di Venezia*. Il campo `nome` funziona bene solo quando il titolo segue una convenzione: con `nome inferno` (titoli *Inferno - Canto …*) tutte le metriche valgono 1, mentre `nome doge` trova solo gli 11 titoli con il disambiguatore *(doge)*, di cui 10 dogi veneziani e uno genovese.
- **Errori tipici.** Per `canto Inferno Dante` i falsi positivi sono canti del Purgatorio e del Paradiso, che citano Dante e l'Inferno: sono vicini al tema ma non rispondono al bisogno informativo. `pasta` restituisce formaggi (*a pasta semicotta*, *pasta filata*): l'ambiguità è nella lingua, non nell'analyzer. `"formaggio a pasta"` non trova nulla perché i 6 formaggi presenti non contengono quella frase.

6.4 Il problema della recall: giudizi incompleti

Per calcolare la recall bisogna conoscere *tutti* i documenti rilevanti. Con 1980 documenti e 50 query ci sarebbero 99 000 coppie da giudicare a mano, cosa impraticabile. Per questo si è usata l'appartenenza alle categorie di Wikipedia, che però è una ground truth **incompleta**: un documento pertinente che non sta nella categoria conta come errore. Per stimare l'effetto si è usata l'idea del *pooling* delle campagne TREC: si sono giudicati a mano i documenti tra i primi 10 di ogni query che il manifest considera non rilevanti (`eval/giudizi_manuali.tsv`).

- Sono **44** documenti in tutto. **6** sono in realtà pertinenti e si concentrano sulle query sui database: *Data Language One*, *Clipper*, *MUMPS*, *PL/SQL*, *PowerBuilder*, *ISI Web of Knowledge*. Wikipedia li classifica in *Linguaggi di programmazione* e non in *Basi di dati*. Con questi giudizi la P@10 di q11 passa da 0,7 a 1,0, q39 da 0,8 a 0,9, q40 da 0,6 a 0,8, e la **P@10 media da 0,878 a 0,890**.
- Gli altri 38 sono errori veri del sistema (canti di un'altra cantica, il doge *di Genova* Antoniotto Adorno per `nome doge`, formaggi per `pasta`…).
- Esistono anche rilevanti non giudicati *fuori* dai primi 10. Per esempio *Temi (astronomia)*, un satellite di Saturno (poi rivelatosi inesistente), sta in un'altra categoria. Per questo la recall misurata è una **stima**: corretta come confronto tra configurazioni, perché tutte sono giudicate con la stessa ground truth, ma non in valore assoluto. Anche la P@10 va letta come limite inferiore.

7 Test automatici

I test automatici sono **52**, tutti superati (`mvn test`: 40 · `python3 -m unittest discover -s scripts`: 12).

Classe	Test	Cosa verifica
`SearcherTest`	20	Le query di §5 sul dataset piccolo: termine singolo, OR, phrase, ordine della frase, maiuscole, assenza di stemming sul nome, stemming ed elisione sul contenuto, `+` / `-` / `AND`, ordinamento BM25, stopword, termine assente, campo non valido, termini mancanti, sintassi errata, spazi multipli.
`IndexerTest`	7	Token prodotti dai due analyzer (separatori, parentesi dei titoli Wikipedia, stopword, elisione, stemming); file non `.txt` e cartelle ignorati; reindicizzare non duplica i documenti; rimozione solo del `.txt` finale.
`MetricheTest`	9	Ogni metrica su un ranking di esempio con valori calcolati a mano; ranking perfetto (tutte = 1); casi limite (ranking vuoto, nessun rilevante: niente divisioni per zero).

`ValutazioneTest`	4	Lettura di manifest e query; ground truth come unione di più argomenti; valutazione end-to-end sul dataset piccolo con giudizi scritti a mano (es. `contenuto ricerca`: precision 1 ma recall 1/5); calcolo della MAP.
`test_build_dataset.py`	12	Conversione wikitext → testo: link, grassetto, template annidati, note, tabelle, immagini e categorie, titoli ed elenchi, sezioni finali, link esterni, redirect, commenti; nomi di file validi.

8 Conclusioni e limiti

- Su 1980 documenti reali l'indicizzazione scala linearmente (circa 2500 documenti/s) e le query rispondono in meno di un millisecondo.
- Le metriche confermano la scelta dell'`ItalianAnalyzer` per il contenuto (MAP 0,720 contro 0,687) e mostrano il compromesso tra query OR (alta recall) e phrase query o query sul nome (alta precision, bassa recall).
- **Limiti.** La ground truth deriva dalle categorie ed è incompleta (§6.4). Le query le ho scritte io, conoscendo gli argomenti, quindi possono essere più "facili" di quelle di un utente reale. Il convertitore wikitext è semplificato: alcuni template che producono testo vengono eliminati: in qualche voce restano parole attaccate (*dellInstitute*) o frasi incomplete. Infine la collezione è limitata a 1980 voci per i limiti di richieste dell'API; lo script può scaricarne di più (`--max-voci`).

Appendice A — Le 50 query di valutazione

Elenco completo delle query di `eval/queries.tsv` con le metriche per query ottenute con l'`ItalianAnalyzer` (configurazione scelta). Le ultime tre colonne riportano l'AP delle tre configurazioni del campo `contenuto` (It = `ItalianAnalyzer`, NoSt = italiano senza stemming, Std = `StandardAnalyzer`), per confrontarle query per query. *Arg.* sono gli argomenti rilevanti (codici della tabella §2.2), |R| il numero di documenti rilevanti, *Rest.* il numero di documenti restituiti.

Id	Query	Arg.	|R|	Rest.	P@10	R@100	Recall	R-Prec	RR	nDCG@10	AP It	AP NoSt	AP Std
q01	`contenuto imperatore romano`	IMP	102	390	1,000	0,833	0,990	0,853	1,000	1,000	0,923	0,873	0,940
q02	`contenuto satellite Giove`	GIO	72	190	1,000	1,000	1,000	0,986	1,000	1,000	0,999	0,992	0,992
q03	`contenuto satellite Saturno`	SAT	61	163	1,000	1,000	1,000	0,967	1,000	1,000	0,996	0,993	0,994
q04	`contenuto satellite naturale pianeta gigante`	GIO, SAT	133	369	1,000	0,737	0,992	0,932	1,000	1,000	0,978	0,945	0,945
q05	`contenuto canto Inferno Dante`	INF	34	178	0,600	1,000	1,000	0,588	1,000	0,691	0,647	0,646	0,400
q06	`contenuto canto "Divina Commedia"`	INF, PUR, PAR	100	156	1,000	0,940	1,000	0,940	1,000	1,000	0,967	0,963	0,961
q07	`contenuto +canto +Paradiso`	PAR	33	68	0,400	1,000	1,000	0,636	1,000	0,466	0,599	0,600	0,603
q08	`contenuto motore di ricerca web`	MOT	73	339	1,000	1,000	1,000	0,932	1,000	1,000	0,983	0,975	0,974
q09	`contenuto linguaggio di programmazione`	LIN	105	226	1,000	0,867	1,000	0,905	1,000	1,000	0,972	0,965	0,968
q10	`contenuto algoritmo di ordinamento`	ALG	27	77	1,000	0,963	0,963	0,963	1,000	1,000	0,937	0,948	0,949
q11	`contenuto database`	DB	28	62	0,700	0,750	0,750	0,500	0,333	0,556	0,395	0,395	0,398
q12	`contenuto doge Venezia`	DOG	127	235	1,000	0,780	0,976	0,921	1,000	1,000	0,960	0,960	0,960
q13	`contenuto pittore Rinascimento`	PIT	133	151	1,000	0,654	0,820	0,789	1,000	1,000	0,758	0,679	0,681
q14	`contenuto battaglia Annibale Cartagine`	PUN	27	248	1,000	1,000	1,000	0,926	1,000	1,000	0,977	0,979	0,980
q15	`contenuto elemento chimico tavola periodica`	ELE	53	411	1,000	1,000	1,000	0,962	1,000	1,000	0,994	0,985	0,965
q16	`contenuto formaggio latte stagionatura`	FOR	6	48	0,600	1,000	1,000	0,833	1,000	0,971	0,915	0,948	0,948
q17	`contenuto pasta`	PAS	9	15	0,600	0,889	0,889	0,667	1,000	0,777	0,811	0,811	0,811
q18	`contenuto "imperatore romano"`	IMP	102	37	0,800	0,324	0,324	0,324	1,000	0,842	0,276	0,227	0,134
q19	`nome inferno`	INF	34	34	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000	1,000
q20	`nome astronomia`	GIO, SAT	133	94	1,000	0,707	0,707	0,707	1,000	1,000	0,707	0,707	0,707
q21	`nome doge`	DOG	127	11	0,900	0,079	0,079	0,079	1,000	0,936	0,078	0,078	0,078
q22	`contenuto +imperatore +impero +augusto`	IMP	102	71	1,000	0,627	0,627	0,627	1,000	1,000	0,621	0,591	0,235
q23	`contenuto cesare augusto senato legioni`	IMP	102	256	1,000	0,794	0,961	0,794	1,000	1,000	0,882	0,869	0,868
q24	`contenuto "satellite di Giove"`	GIO	72	14	0,900	0,181	0,181	0,181	1,000	0,905	0,164	0,105	0,105
q25	`contenuto luna gioviana orbita`	GIO	72	198	1,000	0,472	0,847	0,417	1,000	1,000	0,475	0,371	0,362
q26	`contenuto "satellite naturale di Saturno"`	SAT	61	36	1,000	0,590	0,590	0,590	1,000	1,000	0,590	0,557	0,557
q27	`contenuto Cassini Saturno anelli`	SAT	61	128	1,000	1,000	1,000	0,803	1,000	1,000	0,886	0,898	0,898
q28	`contenuto +canto +Inferno -Purgatorio -Paradiso`	INF	34	19	1,000	0,471	0,471	0,471	1,000	1,000	0,461	0,461	0,219
q29	`contenuto canto Purgatorio`	PUR	33	154	1,000	1,000	1,000	0,818	1,000	1,000	0,920	0,920	0,920
q30	`contenuto anime espiazione monte Purgatorio`	PUR	33	222	0,900	1,000	1,000	0,848	1,000	0,934	0,911	0,911	0,884
q31	`contenuto canto Paradiso Beatrice`	PAR	33	177	0,500	1,000	1,000	0,667	1,000	0,508	0,655	0,655	0,658

q32	contenuto Dante Virgilio	INF, PUR, PAR	100	135	1,000	0,980	1,000	0,980	1,000	1,000	0,994	0,993	0,992
q33	contenuto "motore di ricerca"	MOT	73	68	1,000	0,904	0,904	0,904	1,000	1,000	0,897	0,771	0,769
q34	contenuto indicizzazione pagine web ricerca	MOT	73	339	1,000	0,890	1,000	0,836	1,000	1,000	0,874	0,884	0,906
q35	contenuto "linguaggio di programmazione"	LIN	105	87	1,000	0,781	0,781	0,781	1,000	1,000	0,778	0,761	0,762
q36	contenuto compilatore sintassi programma	LIN	105	211	1,000	0,695	0,800	0,724	1,000	1,000	0,742	0,689	0,689
q37	contenuto ordinamento array confronto	ALG	27	232	1,000	0,963	0,963	0,889	1,000	1,000	0,922	0,929	0,931
q38	contenuto "algoritmo di ordinamento"	ALG	27	24	1,000	0,889	0,889	0,889	1,000	1,000	0,889	0,815	0,741
q39	contenuto "base di dati"	DB	28	13	0,800	0,321	0,321	0,321	1,000	0,870	0,315	0,315	0,318
q40	contenuto DBMS SQL tabelle query	DB	28	69	0,600	0,250	0,250	0,250	0,500	0,534	0,142	0,152	0,162
q41	contenuto "doge di Venezia"	DOG	127	8	0,700	0,055	0,055	0,055	1,000	0,801	0,055	0,055	0,047
q42	contenuto doge Serenissima dogado	DOG	127	147	1,000	0,764	0,969	0,929	1,000	1,000	0,956	0,956	0,955
q43	contenuto pittore affreschi pala d'altare Quattrocento	PIT	133	214	1,000	0,677	0,970	0,857	1,000	1,000	0,889	0,860	0,869
q44	contenuto "pittore italiano"	PIT	133	5	0,300	0,023	0,023	0,023	0,333	0,290	0,011	0,006	0,006
q45	contenuto "seconda guerra punica"	PUN	27	20	0,900	0,630	0,630	0,630	1,000	0,922	0,585	0,585	0,585
q46	contenuto Annibale Scipione assedio	PUN	27	123	1,000	1,000	1,000	0,926	1,000	1,000	0,990	0,995	0,981
q47	contenuto isotopi numero atomico elementi	ELE	53	542	1,000	1,000	1,000	0,981	1,000	1,000	0,992	0,993	0,993
q48	contenuto "formaggio a pasta"	FOR	6	0	0,000	0,000	0,000	0,000	0,000	0,000	0,000	0,000	0,000
q49	contenuto pasta grano duro sugo	PAS	9	77	0,700	1,000	1,000	0,778	1,000	0,825	0,849	0,849	0,849
q50	contenuto indice ricerca informazioni documenti	MOT, DB	101	563	1,000	0,584	0,901	0,584	1,000	1,000	0,687	0,711	0,707
	media				**0,878**	**0,741**	**0,792**	**0,699**	**0,943**	**0,897**	**0,720**	**0,707**	**0,687**